

B-Trees

B-Trees

- Considerations for disk-based storage systems.
 - Why?

Why B-Tree?

- Index structures for large datasets cannot be stored in main memory
- Storing it on disk requires different approach to efficiency
 - disk is mechanical.
 - Its speed depends largely on the time it takes to spin the disk and to move a disk head. Many disks spin at 7,200 RPM.
 - Thus in 1 min, it makes 7,200 revolutions;
 - hence, one revolution occurs in $1/120$ of a second, or 8.3 ms.

Why B-Tree?

- Assuming we want to access 10 million of driving records; that each key is 32 bytes (representing a name) and a record is 256 bytes
- We assume this does not fit in main memory and we are 1 of 20 users on a system (so we have $1/20$ of resources)
- Thus, in 1 sec, we can execute billions of instructions or perform six disk accesses

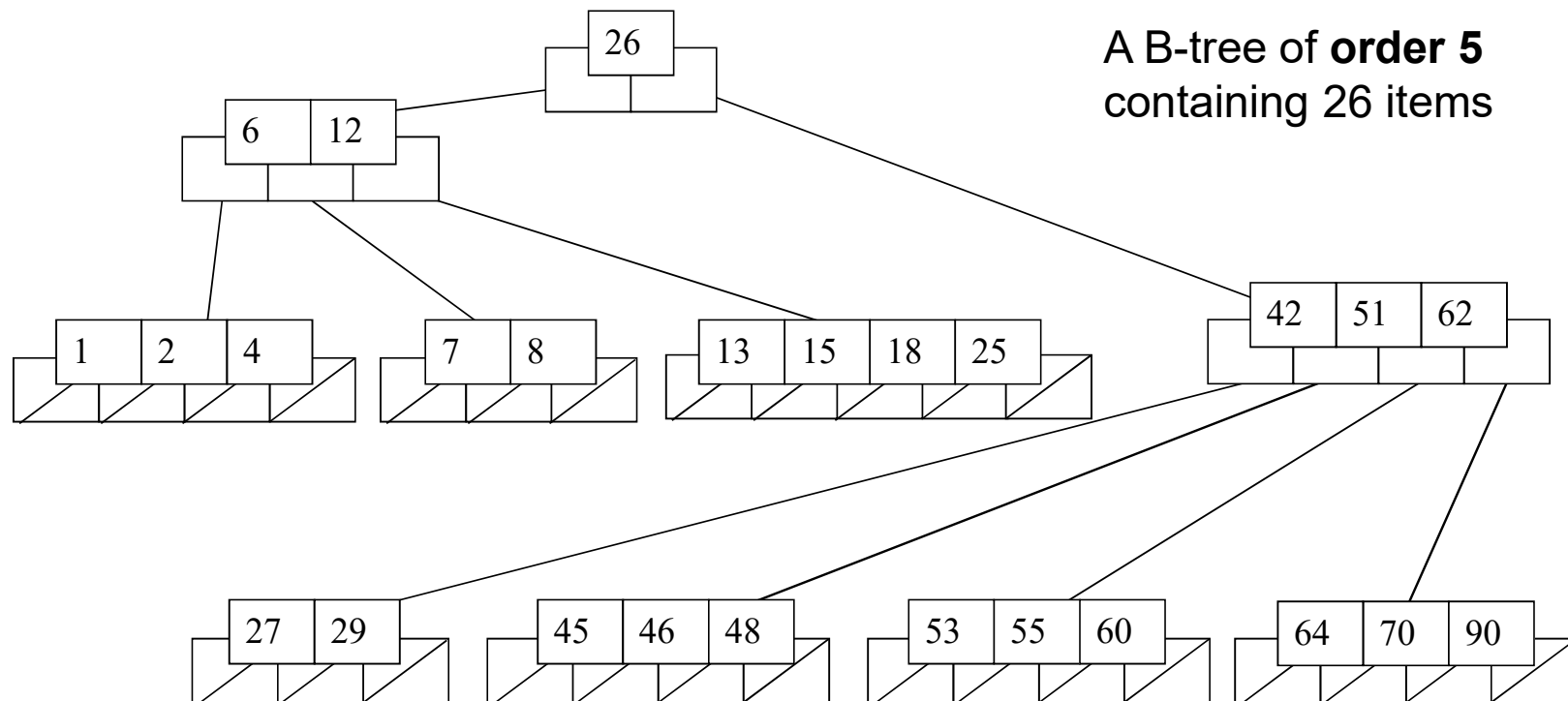
Why B-Tree?

- Unbalanced binary search tree worst case has linear depth thus could require 10 million disk accesses
 - On average, a successful search would require $1.38 \log N$ disk accesses, since $10000000 \approx 2^{24}$
 - an average search would require 32 disk accesses, or 5 sec
- An AVL tree is somewhat better.
 - The worst case of $1.44 \log N$ is unlikely to occur;
 - Thus an AVL tree would use about 25 disk accesses on average, requiring 4 sec.

Algorithms of a B-tree

- A B-tree of order m is an m -way tree (i.e., a tree where each node may have up to m children) in which:
 1. the number of keys in each non-leaf node is one less than the number of its children and these keys partition the keys in the children in the fashion of a search tree
 2. all leaves are on the same level
 3. all non-leaf nodes except the root have at least $\lceil m / 2 \rceil$ children
 4. the root is either a leaf node, or it has from two to m children
 5. a leaf node contains no more than $m - 1$ keys
- The number m should always be odd

An example B-Tree



Note that all the leaves are at the same level

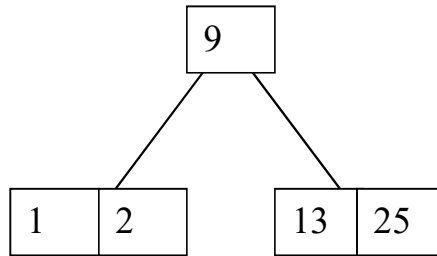
Constructing a B-tree

- Suppose we start with an empty B-tree and keys arrive in the following order: 1 13 9 2 25 6 14 28 17 7 52 16 48 68 3 26 29 53 55 45
- We want to construct a B-tree of order 5
- The first four items go into the root:

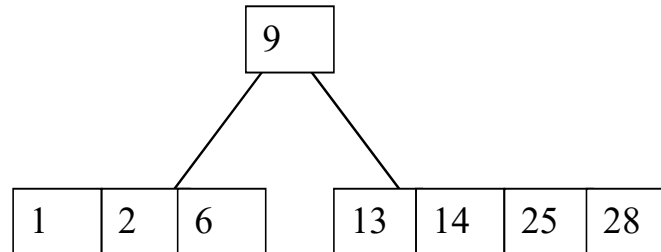
1	2	9	13
---	---	---	----

- To put the fifth item in the root would violate condition 5
- Therefore, when 25 arrives, pick the middle key to make a new root

Constructing a B-tree (contd.)

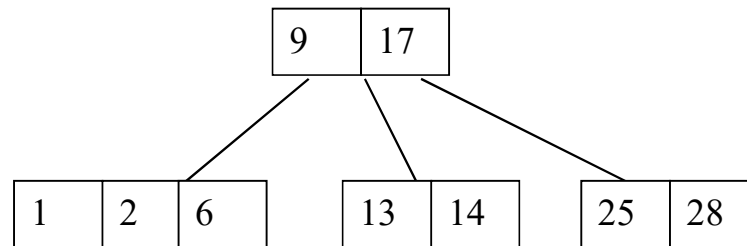


6, 14, 28, 17 get added to the leaf nodes:

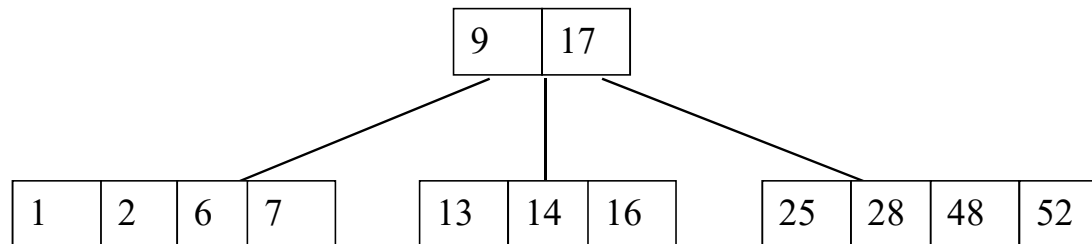


Constructing a B-tree (contd.)

Adding 17 to the right leaf node would over-fill it, so we take the middle key, promote it (to the root) and split the leaf



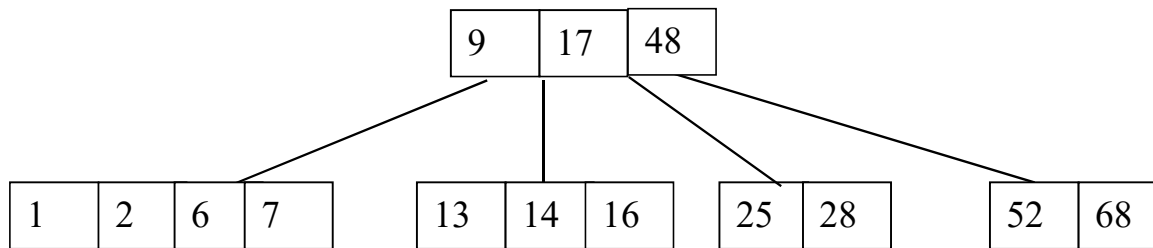
7, 52, 16, 48 get added to the leaf nodes



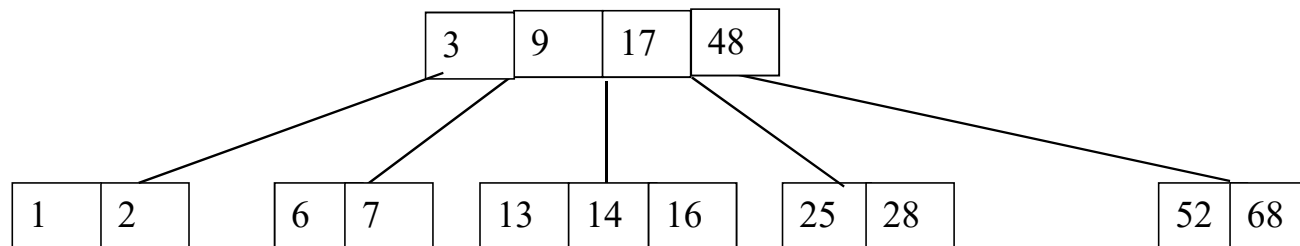
Constructing a B-tree (contd.)

Adding 68, 3

Adding 68 causes us to split the right most leaf, promoting 48 to the root

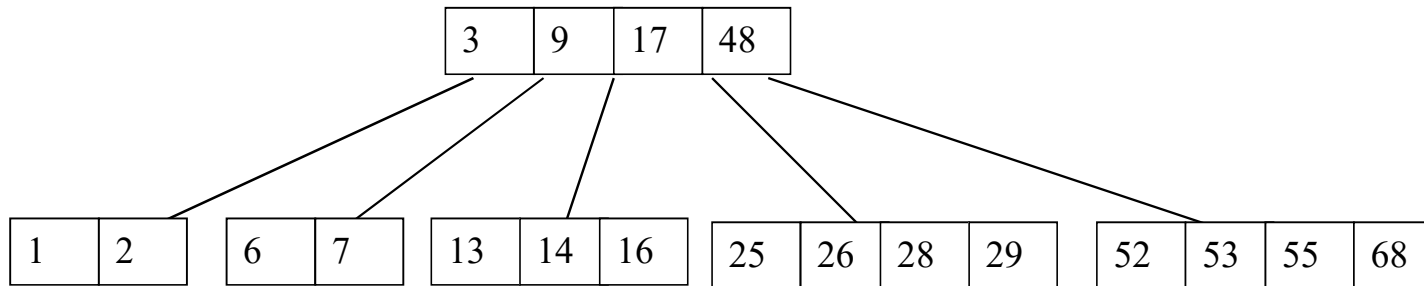


adding 3 causes us to split the left most leaf, promoting 3 to the root



Constructing a B-tree (contd.)

Adding 26, 29, 53, 55 then go into the leaves

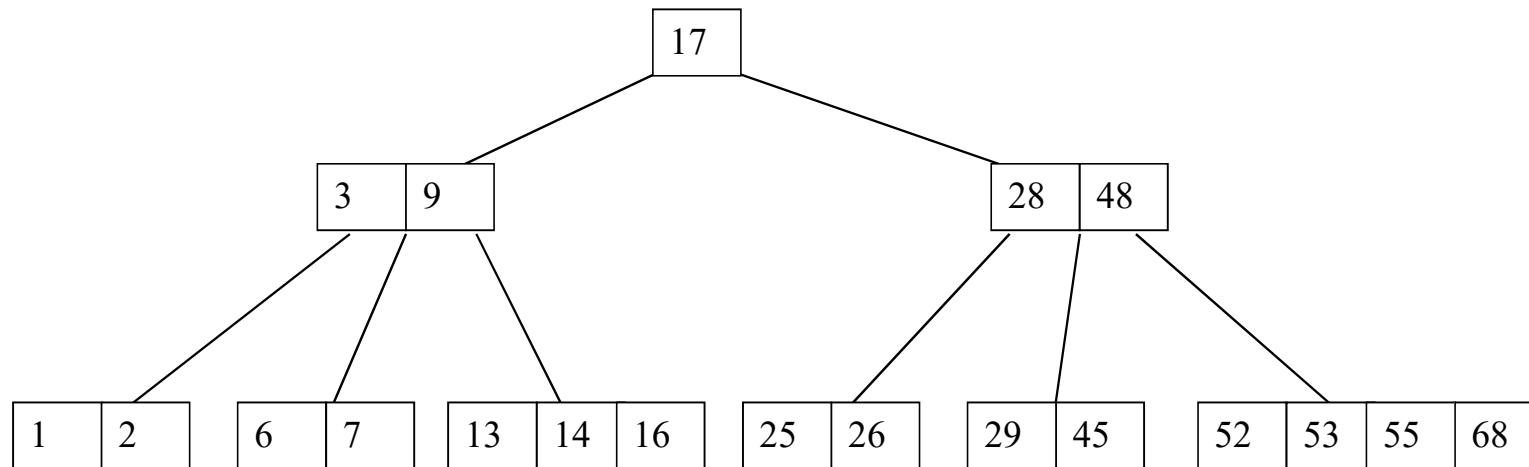


Adding 45 causes a split of

25	26	28	29
----	----	----	----

and promoting 28 to the root then causes the root to split

Constructing a B-tree (contd.)



Inserting into a B-Tree

- Attempt to insert the new key into a leaf
- If this would result in that leaf becoming too big, split the leaf into two, promoting the middle key to the leaf's parent
- If this would result in the parent becoming too big, split the parent into two, promoting the middle key
- This strategy might have to be repeated all the way to the top
- If necessary, the root is split in two and the middle key is promoted to a new root, making the tree one level higher

Exercise

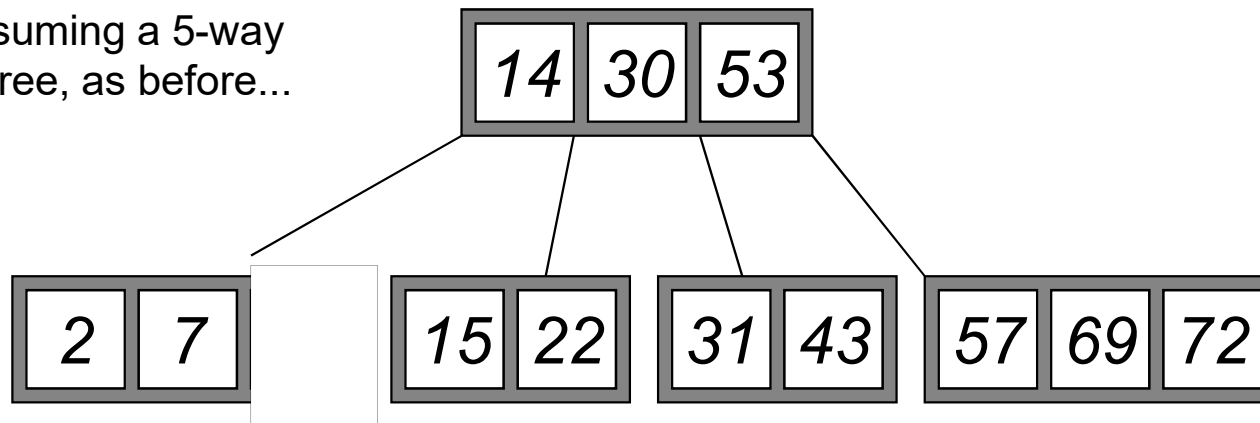
- Create a 5-way B-tree:
- 3, 7, 59, 32, 45, 1, 5, 14, 25, 24, 13, 11, 8, 19, 61, 75, 76, 80, 77, 81, 99, 60

Removal from a B-tree

- There are three possible ways we can remove *from* a leaf. :
 1. Delete only if the key is a **leaf node** and doesn't cause that leaf node to have **too few keys**
 2. If the key is *not* in a leaf, promote the predecessor or successor key to the non-leaf deleted key's position after deleted the key.
 3. If (1) or (2) lead to a leaf node containing less than the minimum number of keys then:
 - if one of them has more than the min. number of keys then we can promote one of its keys to the parent and take the parent key into our lacking leaf
 - if neither of them has more than the min. number of keys then the lacking leaf and one of its neighbours can be combined with their shared parent (the opposite of promoting a key) and the new leaf will have the correct number of keys; if this step leave the parent with too few keys then we repeat the process up to the root itself, if required

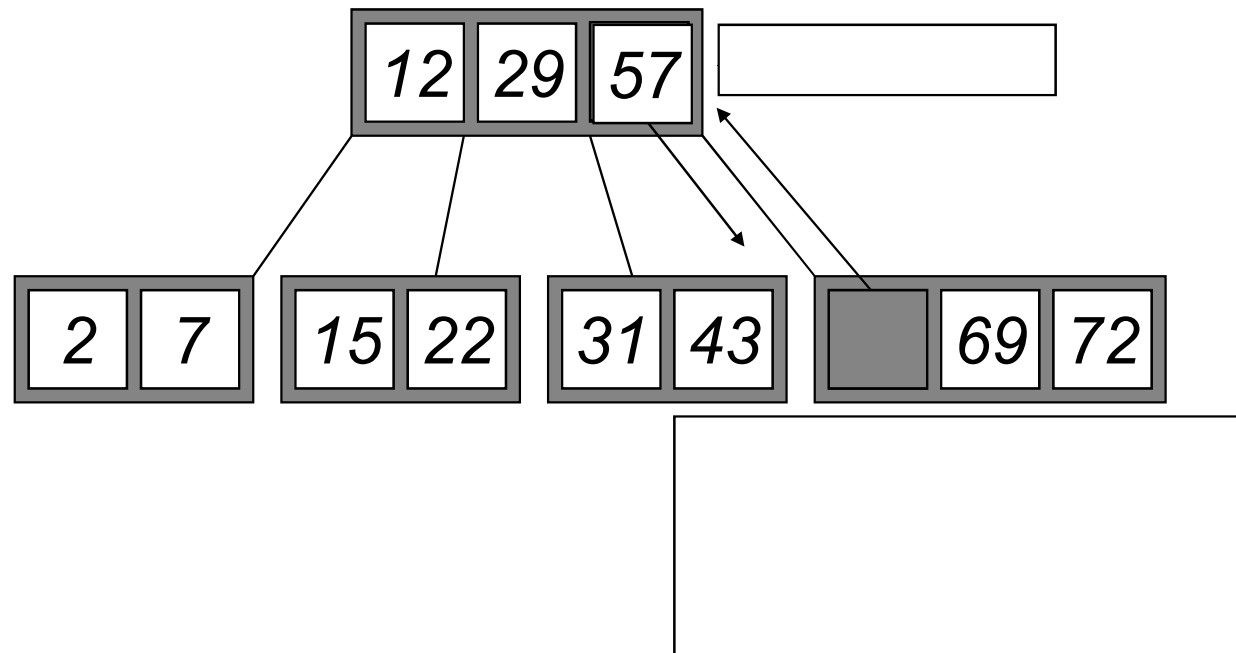
Type #1: Simple leaf deletion

Assuming a 5-way
B-Tree, as before...

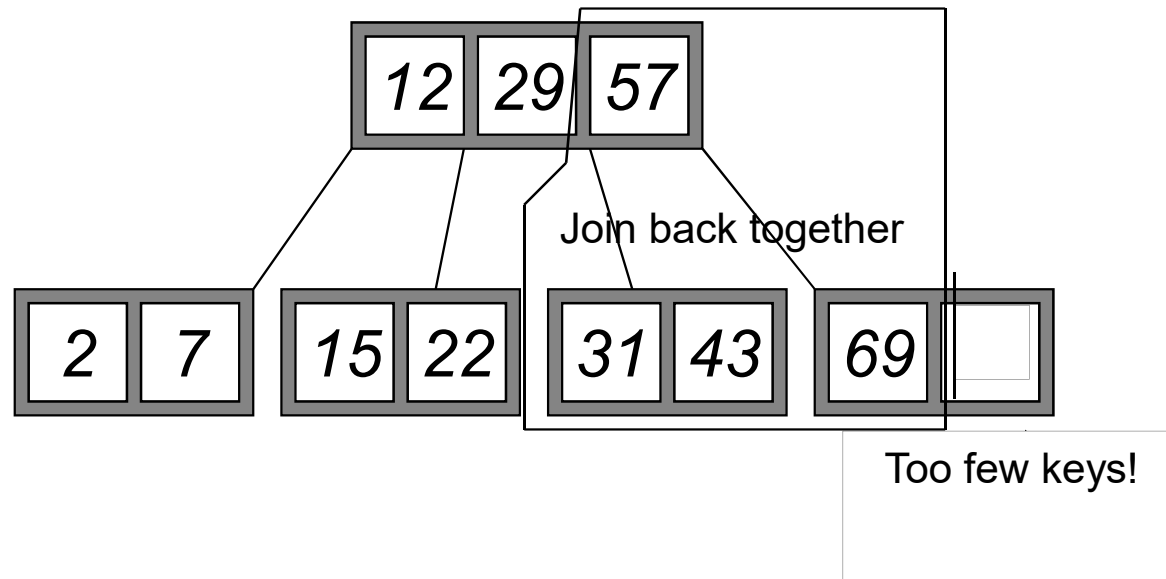


Delete 9: Since there are enough
keys in the node, just delete it

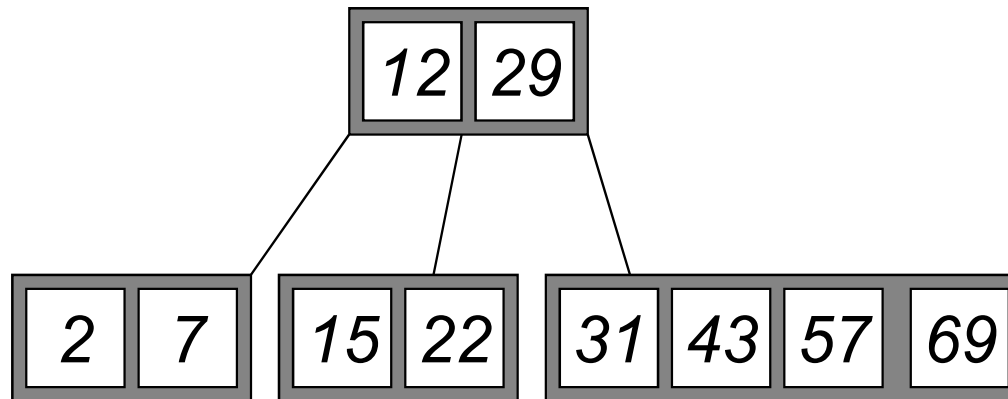
Type #2: Simple non-leaf deletion



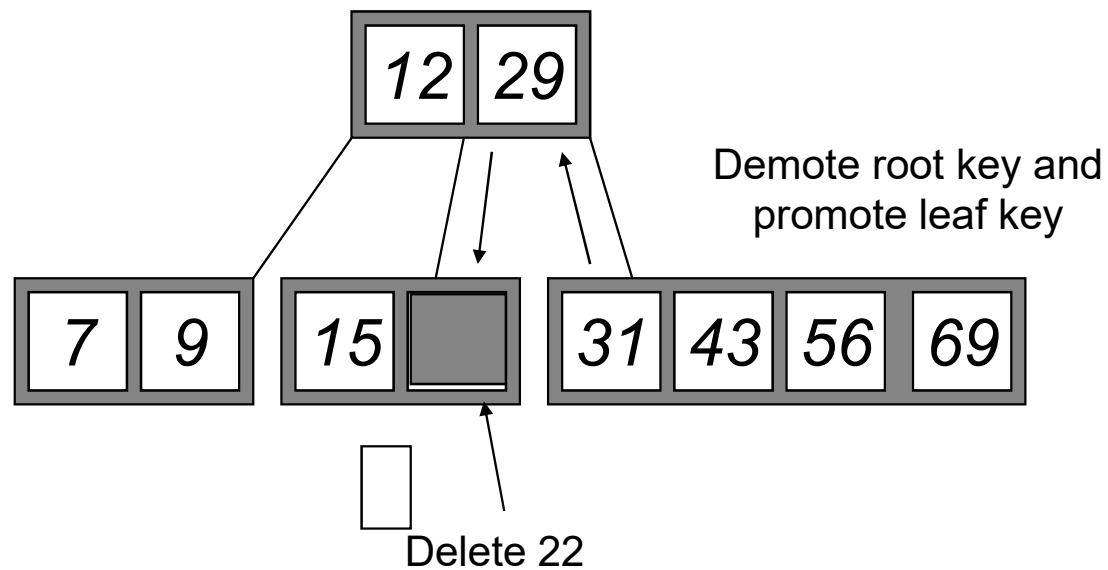
Type #4: Too few keys in node and its siblings



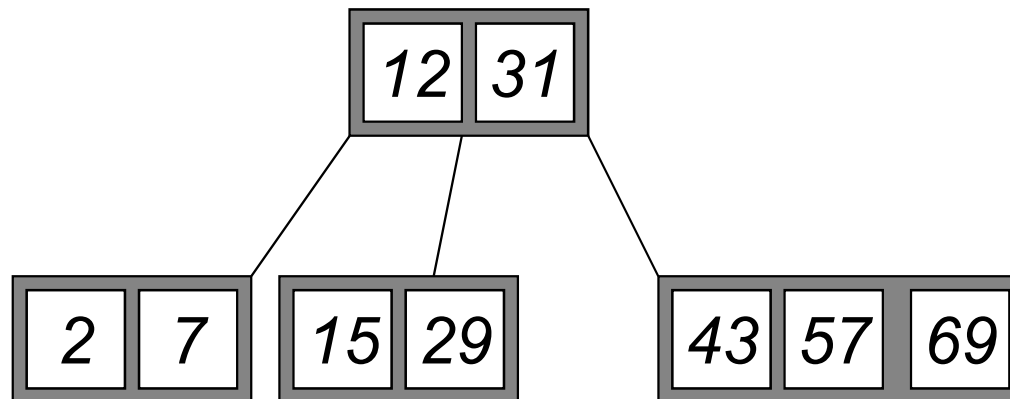
Type #4: Too few keys in node and its siblings



Type #3: Enough siblings



Type #3: Enough siblings



Exercise in Removal from a B-Tree

- Given 5-way B-tree created by these data (previous exercise):
- 3, 7, 59, 32, 45, 1, 5, 14, 25, 24, 13, 11, 8, 19, 61, 75, 76, 80, 77, 81, 99, 60
- Delete these keys: 60, 77, 99

Analysis of B-Trees

- The maximum number of items in a B-tree of order m and height h :

root	$m - 1$
level 1	$m(m - 1)$
level 2	$m^2(m - 1)$
...	
level h	$m^h(m - 1)$

- So, the total number of items is

$$(1 + m + m^2 + m^3 + \dots + m^h)(m - 1) =$$
$$[(m^{h+1} - 1) / (m - 1)] (m - 1) = \mathbf{m^{h+1} - 1}$$

- When $m = 5$ and $h = 2$ this gives $5^3 - 1 = 124$

Reasons for using B-Trees

- When searching tables held on disc, the cost of each disc transfer is high but doesn't depend much on the amount of data transferred, especially if consecutive items are transferred
 - If we use a B-tree of order 101, say, we can transfer each node in one disc read operation
 - A B-tree of order 101 and height 3 can hold $101^4 - 1$ items (approximately 100 million) and any item can be accessed with 3 disc reads (assuming we hold the root in memory)
- If we take $m = 3$, we get a **2-3 tree**, in which non-leaf nodes have two or three children (i.e., one or two keys)
 - B-Trees are always balanced (since the leaves are all at the same level), so 2-3 trees make a good type of balanced tree

Comparing Trees

- Binary trees
 - Can become *unbalanced* and *lose* their good time complexity (big O)
 - AVL trees are strict binary trees that *overcome the balance problem*
 - Heaps remain balanced but only *prioritise* (not order) the keys
- Multi-way trees
 - B-Trees can be *m*-way, they can have any (odd) number of children
 - One B-Tree, the 2-3 (or 3-way) B-Tree, *approximates* a permanently balanced binary tree, exchanging the AVL tree's balancing operations for insertion and (more complex) deletion operations